

Relatório do Projeto Prático - Vigilância de Partições Retangulares

Unidade Curricular: Métodos de Apoio à Decisão

Alunos: Orlando Soares, Maximiliano Sá, Paulo Lin

Ano Letivo: 2025/2026

1. Introdução

O presente relatório descreve a resolução do projeto prático proposto na unidade curricular de Métodos de Apoio à Decisão, subordinado ao tema **Vigilância de Partições Retangulares**.

O problema em estudo consiste em determinar a colocação ótima de guardas em vértices de uma partição retangular, de forma a assegurar a cobertura visual de todos os retângulos da partição, ou apenas de um subconjunto destes, minimizando simultaneamente o número de guardas utilizados.

Este problema insere-se no domínio da **otimização combinatória**, apresentando semelhanças diretas com o problema clássico de *Set Covering*, no qual se pretende selecionar o menor número de conjuntos capazes de cobrir um universo de elementos.

Neste contexto:

- cada vértice da partição corresponde a uma possível localização para um guarda;
- cada guarda cobre um conjunto de retângulos incidentes;
- o objetivo global é encontrar a combinação mínima de vértices cuja união de coberturas satisfaça os requisitos de vigilância.

Ao longo do trabalho foram exploradas múltiplas metodologias de resolução, permitindo comparar paradigmas heurísticos e exatos de apoio à decisão.

Foram estudadas as seguintes abordagens:

- Estratégias Greedy;
- Programação Inteira Binária;
- Programação por Restrições com MAC e AC-3;
- Resolução em Google OR-Tools;

- Programação Dinâmica;
- Extensões do problema com guardas coloridos e guardas de alcance ampliado.

2. Modelação Formal do Problema

Considere-se uma partição retangular $\Pi = \{R_1, R_2, \dots, R_n\}$, composta por n retângulos, e um conjunto de vértices geométricos $V = \{v_1, v_2, \dots, v_m\}$.

Cada vértice pode potencialmente receber um guarda.

Um guarda colocado num vértice v_i vigia todos os retângulos incidentes a esse vértice.

2.1 Matriz de Cobertura

Define-se uma matriz binária de cobertura $A \in \{0, 1\}^{m \times n}$:

$$A[i][j] = \begin{cases} 1, & \text{se o vértice } v_i \text{ cobre o retângulo } R_j \\ 0, & \text{caso contrário} \end{cases}$$

Esta matriz constitui a representação central utilizada por todos os algoritmos implementados.

2.2 Objetivo Global

Pretende-se determinar um subconjunto mínimo de vértices:

$$G \subseteq V$$

que satisfaça:

$$\bigcup_{v \in G} \text{covers}(v) \supseteq \Pi'$$

onde Π' representa o conjunto de retângulos a vigiar.

2.3 Cobertura Parcial

O modelo suporta igualmente o caso em que apenas um subconjunto $\Pi' \subset \Pi$ tem de ser coberto. Neste caso, os algoritmos recebem um parâmetro `required` com os índices dos retângulos obrigatórios:

$$\Pi' = \{R_j \mid j \in \text{required}\}$$

Quando `required` é omitido, assume-se $\Pi' = \Pi$ (cobertura total), preservando o comportamento original. Esta generalização foi implementada em todos os solvers sem alterar a interface existente.

3. Estratégias Greedy

3.1 Motivação

As estratégias greedy foram utilizadas como primeira abordagem heurística devido à sua simplicidade de implementação e reduzido custo computacional.

Estas heurísticas constroem a solução incrementalmente, tomando em cada passo a decisão local aparentemente mais vantajosa.

3.2 Heurística de Máxima Cobertura Residual

Em cada iteração é selecionado o vértice que cobre o maior número de retângulos ainda não vigiados.

Pseudocódigo

```
GreedyCoverage(U, V):
  guards <- {}
  while U não vazio:
    escolher v em V que maximiza |covers(v) ∩ U|
    guards <- guards ∪ {v}
    U <- U \ covers(v)
  return guards
```

3.3 Complexidade

Sejam:

- m o número de vértices;

- n o número de retângulos.

A cada iteração é necessário avaliar a cobertura residual de todos os vértices, resultando numa complexidade aproximada:

$$O(m \cdot n)$$

por iteração, conduzindo na prática a tempos de execução muito baixos.

3.4 Discussão

A abordagem greedy revelou-se extremamente rápida e adequada para produzir soluções iniciais. Contudo, por não considerar consequências globais futuras, conduz frequentemente a soluções subótimas.

4. Programação Inteira Binária

4.1 Formulação Matemática

Foi modelado o problema como um programa linear inteiro binário.

Variáveis de decisão

$$x_i = \begin{cases} 1, & \text{se existe guarda no vértice } i \\ 0, & \text{caso contrário} \end{cases}$$

Função objetivo

$$\min \sum_{i=1}^m x_i$$

Restrições de cobertura

Para cada retângulo R_j :

$$\sum_{i: A[i][j]=1} x_i \geq 1$$

Cada retângulo deve ser coberto por pelo menos um guarda.

4.2 Complexidade

A formulação é NP-difícil por equivalência ao problema de Set Cover.

No entanto, solvers modernos conseguem resolver eficientemente instâncias de dimensão moderada.

4.3 Implementação

A implementação foi realizada em Python utilizando o solver CP-SAT do Google OR-Tools, que permite:

- criação de variáveis binárias;
- adição de restrições lineares;
- otimização da função objetivo.

5. Programação por Restrições, MAC com AC-3

5.1 Formulação CSP

Cada vértice é modelado como uma variável booleana:

$$X_i \in \{0, 1\}$$

Para cada retângulo gera-se uma restrição disjuntiva:

$$X_a \vee X_b \vee X_c \vee X_d$$

onde a, b, c, d representam os vértices incidentes.

5.2 Backtracking com MAC

Foi implementado um algoritmo de procura em profundidade com:

- seleção de variáveis por MRV;
- atribuição binária;
- manutenção de consistência de arcos após cada decisão.

5.3 Algoritmo AC-3

Após cada atribuição é executado o AC-3 para eliminar valores inconsistentes dos domínios das restantes variáveis.

Pseudocódigo

```
AC3(queue):
    enquanto queue não vazio:
        remover arco (Xi, Xj)
        se Revise(Xi, Xj):
            se domínio(Xi) vazio -> falha
            adicionar arcos vizinhos

Reise(Xi, Xj):
    revisado = falso
    para cada valor a em domínio(Xi):
        se não existe valor b em domínio(Xj) que satisfaça a restrição entre Xi
e Xj com (a, b):
            remover a de domínio(Xi)
            revisado = verdadeiro
    retornar revisado
```

5.4 Vantagens

Comparativamente ao backtracking puro, a propagação MAC permitiu uma redução substancial do espaço de procura.

6. Resolução Declarativa em Google OR-Tools

Foi implementada uma solução industrial com OR-Tools, utilizando o módulo CP-SAT.

Este solver demonstrou excelente eficiência na obtenção da solução ótima, apresentando tempos de execução residuais.

7. Programação Dinâmica

7.1 Definição do Estado

Seja S um subconjunto de retângulos já cobertos.

Define-se:

$$DP[S] = \text{número mínimo de guardas necessários para cobrir } S$$

7.2 Transição

Para cada vértice v :

$$DP[S \cup \text{covers}(v)] = \min(DP[S \cup \text{covers}(v)], DP[S] + 1)$$

7.3 Análise

Esta abordagem garante a obtenção da solução ótima.

Todavia, apresenta complexidade:

$$O(2^n \cdot m)$$

sendo por isso apenas prática para instâncias pequenas ou médias.

8. Extensões do Problema

8.1 Guardas com cores (coloração de conflitos)

Nesta extensão introduz-se uma restrição adicional de compatibilidade entre guardas. Em particular, dois guardas não podem partilhar a mesma cor caso exista pelo menos um retângulo que seja observado simultaneamente por ambos. Esta condição modela situações em que a presença conjunta de dois guardas no mesmo domínio de vigilância implica conflito, exigindo a sua diferenciação através de cores.

Após a obtenção de uma solução mínima para o problema original de colocação de guardas, constrói-se um grafo de conflitos $G = (V, E)$, onde:

- cada vértice representa um guarda colocado;
- existe uma aresta entre dois guardas se estes partilham a vigilância de pelo menos um retângulo.

Formalmente, para guardas g_i e g_j , existe uma aresta (g_i, g_j) se:

$$\exists r \in \Pi : r \in C(g_i) \wedge r \in C(g_j)$$

onde $C(g)$ representa o conjunto de retângulos cobertos pelo guarda g .

O problema reduz-se então a um problema clássico de coloração de grafos, cujo objetivo é atribuir cores aos vértices de forma a que vértices adjacentes tenham cores diferentes, minimizando o número total de cores utilizadas.

Foi implementado um algoritmo de coloração exata por backtracking com poda (branch-and-bound), que explora sistematicamente as atribuições possíveis de cores e elimina ramos cuja solução parcial já excede a melhor solução encontrada.

Complexidade

O problema de coloração mínima de grafos é NP-completo. O algoritmo implementado tem complexidade exponencial no pior caso:

$$O(k^n)$$

onde n é o número de guardas e k o número de cores utilizadas na solução corrente. No entanto, a ordenação dos vértices por grau e a poda por limite superior reduzem significativamente o espaço de pesquisa na prática.

8.2 Guardas com maior alcance

Nesta extensão considera-se um parâmetro de alcance D , que permite a um guarda vigiar não apenas os retângulos diretamente incidentes no seu vértice, mas também retângulos adjacentes até uma distância de grafo no máximo D .

Para modelar esta extensão, foi construído o grafo de adjacência entre retângulos (grafo dual), onde:

- cada nó representa um retângulo;
- existe uma aresta entre dois retângulos se estes partilham pelo menos um vértice.

Formalmente:

$$(r_i, r_j) \in E \iff V(r_i) \cap V(r_j) \neq \emptyset$$

A expansão da cobertura de um guarda é então obtida através de uma pesquisa em largura (BFS) neste grafo, partindo dos retângulos diretamente observados pelo guarda e explorando vizinhos até profundidade D .

O conjunto final de retângulos cobertos por um guarda corresponde à união dos retângulos alcançados pela BFS a partir dos seus retângulos incidentes.

Este conjunto expandido substitui a cobertura original no modelo de otimização, permitindo que o problema original de set cover seja resolvido com uma matriz de incidência modificada.

Complexidade

A construção do grafo de retângulos tem custo:

$$O(n^2 \cdot m)$$

onde n é o número de retângulos e m o número médio de vértices por retângulo.

A BFS para cada guarda tem custo:

$$O(|V| + |E|)$$

Como esta expansão é feita antes da resolução do problema principal, o custo total depende do solver escolhido (tipicamente exponencial para métodos exatos como DP ou ILP), mas a fase de pré-processamento é polinomial.

9. Descrição da Implementação

A implementação foi desenvolvida integralmente em Python, organizando-se da seguinte maneira:

- `main.py`, leitura da partição, construção da matriz de cobertura e print dos resultados do método escolhido;
- `greedy.py`, heurísticas greedy;
- `integer_solver.py`, modelo de programação inteira;
- `csp_mac.py`, backtracking com MAC + AC3;
- `dynamic.py`, programação dinâmica;

- `extensions.py`, guardas coloridos e alcance D.

Foram ainda produzidas implementações paralelas em:

- Google OR-Tools.

Esta modularização facilitou a reutilização da mesma matriz de cobertura por todos os paradigmas de resolução.

10. Resultados Experimentais

Para avaliar o desempenho das diferentes abordagens implementadas, foi desenvolvido um script de benchmarking responsável por executar múltiplas instâncias e calcular o tempo médio de execução de cada algoritmo.

Os testes foram realizados sobre conjuntos de instâncias contendo 10, 30, 50, 100, 500 e 1000 retângulos, com 5 instâncias por ficheiro.

O benchmarking foi implementado utilizando:

- `time.perf_counter()` para medição temporal;
- execução isolada de processos através de `multiprocessing`;
- limite máximo de execução de 60 segundos por algoritmo.

O código executa cada solver múltiplas vezes e calcula a média dos tempos obtidos.

10.1 Resultados Obtidos

Instância	Dynamic Programming	Greedy	Integer Programming	CSP + MAC/AC3
10rect_5instances	0.726390 s	0.786435 s	0.749426 s	0.715588 s
30rect_5instances	Time Limit Exceeded (60 s)	0.719921 s	0.727680 s	0.729817 s
50rect_5instances	Time Limit Exceeded (60 s)	0.661036 s	0.685144 s	0.705793 s
100rect_5instances	Time Limit Exceeded (60 s)	0.779034 s	0.765813 s	0.994237 s
500rect_5instances	Time Limit Exceeded (60 s)	2.397914 s	0.973272 s	Time Limit Exceeded (60 s)
1000rect_5instances	Time Limit Exceeded (60 s)	19.364834 s	1.836622 s	Time Limit Exceeded (60 s)

10.2 Análise dos Resultados

Os resultados experimentais obtidos permitem observar diferenças claras de escalabilidade entre os diferentes paradigmas de resolução implementados.

A **Programação Dinâmica** revelou-se impraticável para além das instâncias mais pequenas: excedeu o limite de 60 segundos a partir das 30 instâncias, confirmando a sua complexidade exponencial $O(2^n \cdot m)$.

O **CSP com MAC+AC3** foi competitivo nas instâncias pequenas e médias (até 100 retângulos, com tempos entre 0.71 s e 0.99 s), mas colapsou nas instâncias de 500 e 1000 retângulos, evidenciando os limites do backtracking em espaços de procura muito grandes.

O **Greedy** manteve-se funcional ao longo de todos os conjuntos, mas evidenciou uma degradação crescente: de ≈ 0.7 s nas instâncias pequenas, passou para 2.4 s com 500 retângulos e 19.4 s com 1000 retângulos. Este comportamento mostra que o custo acumulado das iterações cresce de forma considerável com a dimensão do problema.

A **Programação Inteira com OR-Tools (CP-SAT)** destacou-se como a abordagem mais robusta em todos os conjuntos testados. Resolveu as instâncias de 500 retângulos em 0.97 s e as de 1000 retângulos em 1.84 s — consideravelmente mais rápido do que o Greedy nas instâncias maiores, e garantindo sempre a solução ótima.

No geral, os resultados mostram que:

- **Programação Inteira com OR-Tools** foi a abordagem mais robusta, superando o Greedy em velocidade para instâncias grandes e garantindo optimalidade;
- **Greedy** é adequado para instâncias pequenas ou como solução inicial rápida, mas não escala bem;
- **CSP + MAC/AC3** permanece adequado para instâncias de dimensão moderada;
- **Programação Dinâmica** tem valor sobretudo acadêmico, restringindo-se a instâncias muito pequenas.

11. Conclusão

O presente projeto permitiu estudar e comparar múltiplas metodologias de apoio à decisão aplicadas ao problema de vigilância de partições retangulares, um problema de cobertura combinatória com relevância teórica e prática.

Os resultados experimentais confirmam as expectativas teóricas de forma clara. A **Programação Dinâmica** mostrou-se impraticável além de instâncias muito pequenas, dada a sua complexidade $O(2^n \cdot m)$. O **CSP com MAC+AC3**, embora eficaz em instâncias moderadas, não conseguiu escalar para além dos 100 retângulos. O **Greedy**, apesar de simples e funcional, revelou degradação expressiva nas instâncias maiores — chegando a 19.4 s para 1000 retângulos — mostrando que não constitui a melhor alternativa em termos de velocidade quando a dimensão do problema cresce.

A **Programação Inteira com OR-Tools (CP-SAT)** destacou-se inequivocamente como a abordagem mais robusta: resolveu todos os conjuntos de teste dentro do limite temporal, mantendo tempos abaixo dos 2 segundos mesmo para 1000 retângulos, e garantindo sempre a solução ótima. É a escolha recomendada para instâncias de qualquer dimensão onde se exija qualidade e escalabilidade.

Em suma:

- **OR-Tools (CP-SAT)** — solução preferencial: ótima, rápida e escalável;
- **Greedy** — útil como heurística inicial ou em instâncias pequenas, mas não escala;

- **CSP com MAC+AC3** — adequado para instâncias moderadas, com valor acadêmico relevante;
- **Programação Dinâmica** — referência de optimalidade, restrita a instâncias muito pequenas.

As extensões implementadas — coloração de guardas e alcance ampliado — demonstraram ainda que a abstração central do projeto, a matriz de cobertura, é suficientemente flexível para acomodar variantes mais ricas do problema sem alterar a arquitectura de resolução.

12. Adaptação ao Formato das Instâncias

O gerador de instâncias utilizado no projeto produz ficheiros no formato:

```
k = <int>
<id> <m> x1 y1 x2 y2 ... xm ym
```

onde:

- `k` representa o número de retângulos existentes na instância;
- `id` representa o identificador do retângulo, variando entre `1` e `k`;
- `m` representa o número de vértices do retângulo;
- os pares `(xi, yi)` representam as coordenadas dos vértices.

Exemplo:

```
8
1 4 0 6 4 6 4 7 0 7
2 4 4 6 8 6 8 7 4 7
...
```

Cada linha define um retângulo da partição.

12.1 Parser das Instâncias

Foi desenvolvido um parser responsável por:

- ler múltiplas instâncias no mesmo ficheiro;
- extrair vértices únicos;
- construir os retângulos;
- gerar automaticamente a matriz de cobertura.

```

class Rectangle:
    def __init__(self, rid, vertices):
        self.id = rid
        self.vertices = vertices

class Instance:
    def __init__(self, k):
        self.k = k
        self.rectangles = []

        # ordered unique vertices
        self.vertices = []

def parse_instances(filename):
    with open(filename, 'r') as f:
        raw_lines = [ln.strip() for ln in f if ln.strip()]

    instances = []
    i = 0

    # optional number of instances
    n = None
    if (
        i + 1 < len(raw_lines)
        and len(raw_lines[i].split()) == 1
        and len(raw_lines[i + 1].split()) == 1
    ):
        try:
            n = int(raw_lines[i])
            i += 1
        except ValueError:
            pass

    instances_parsed = 0

    while i < len(raw_lines) and (n is None or instances_parsed < n):

        # instance size k
        if len(raw_lines[i].split()) != 1:
            raise ValueError(
                f"Expected instance grid size k at line {i+1}: '{raw_lines[i]}'"
            )

        k = int(raw_lines[i])
        inst = Instance(k)

```

```
i += 1

# read rectangles
while i < len(raw_lines):

    parts = raw_lines[i].split()

    # next instance starts
    if len(parts) == 1:
        break

    try:
        nums = list(map(int, parts))
    except ValueError:
        raise ValueError(
            f"Non-integer token on line {i+1}: '{raw_lines[i]}'"
        )

    rid = nums[0]
    m = nums[1]
    coords = nums[2:]

    if len(coords) != 2 * m:
        raise ValueError(
            f"Expected {2*m} coord values for face {rid}, "
            f"got {len(coords)} on line {i+1}"
        )

    vertices = []

    for j in range(0, len(coords), 2):
        v = (coords[j], coords[j + 1])

        vertices.append(v)

    # keep unique ordered vertices
    if v not in inst.vertices:
        inst.vertices.append(v)

    inst.rectangles.append(Rectangle(rid, vertices))
    i += 1

    instances.append(inst)
    instances_parsed += 1

return instances
```


Desta forma todos os algoritmos implementados podem ser executados diretamente sobre as instâncias produzidas pelo gerador fornecido.

13. Implementações dos Algoritmos

13.1 Implementação Greedy em Python

```
class GreedySolver:
    def __init__(self, coverage_matrix, required=None):
        self.A = coverage_matrix
        self.num_vertices = len(coverage_matrix)
        self.num_rectangles = len(coverage_matrix[0])
        # required: indices of rectangles that must be covered (None = all)
        self.required = set(required) if required is not None else set(range(self.num_rectangles))

    def solve(self):
        uncovered = set(self.required)
        guards = []

        while uncovered:
            best_vertex = None
            best_cover = set()

            for v in range(self.num_vertices):
                current_cover = {r for r in uncovered if self.A[v][r] == 1 and r in self.required}
                if len(current_cover) > len(best_cover):
                    best_cover = current_cover
                    best_vertex = v

            guards.append(best_vertex)
            uncovered -= best_cover

        return guards
```

13.2 Implementação de Programação Inteira com OR-Tools

```
from ortools.sat.python import cp_model

class IntegerSolver:
    def __init__(self, coverage_matrix, required=None):
        self.A = coverage_matrix
        self.m = len(coverage_matrix)
        self.n = len(coverage_matrix[0])
        # required: indices of rectangles that must be covered (None = all)
        self.required = list(required) if required is not None else list(range(
            self.n))

    def solve(self):
        model = cp_model.CpModel()
        x = [model.NewBoolVar(f'x{i}') for i in range(self.m)]

        for j in self.required:
            model.Add(sum(x[i] for i in range(self.m) if self.A[i][j] == 1) >=
1)

        model.Minimize(sum(x))

        solver = cp_model.CpSolver()
        solver.Solve(model)

        return [i for i in range(self.m) if solver.Value(x[i]) == 1]
```

13.3 Implementação MAC + AC3

```
class MACSolver:
    def __init__(self, coverage_matrix, required=None):
        self.A = coverage_matrix
        self.m = len(coverage_matrix)
        self.n = len(coverage_matrix[0])
        # required: indices of rectangles that must be covered (None = all)
        self.required = list(required) if required is not None else list(range(
            self.n))

    def is_covered(self, assignment, rect):
        return any(assignment[i] == 1 and self.A[i][rect] == 1 for i in range(s
            elf.m))

    def valid_partial(self, assignment):
        for r in self.required:
            possible = False
            for i in range(self.m):
                if assignment[i] is None and self.A[i][r] == 1:
                    possible = True
                if assignment[i] == 1 and self.A[i][r] == 1:
                    possible = True
            if not possible:
                return False
        return True

    def backtrack(self, assignment, idx=0):
        if idx == self.m:
            if all(self.is_covered(assignment, r) for r in self.required):
                return assignment
            return None

        for val in [0,1]:
            assignment[idx] = val
            if self.valid_partial(assignment):
                result = self.backtrack(assignment, idx+1)
                if result:
                    return result
            assignment[idx] = None
        return None

    def solve(self):
        return self.backtrack([None]*self.m)
```

13.4 Implementação de Programação Dinâmica

```
class DPSolver:
    def __init__(self, coverage_sets, n_rectangles, required=None):
        self.coverage_sets = coverage_sets
        self.n = n_rectangles
        # required: indices of rectangles that must be covered (None = all)
        required_indices = required if required is not None else range(n_rectan
gles)

        self.full = 0
        for r in required_indices:
            self.full |= (1 << r)

    def solve(self):
        from collections import deque

        full = self.full
        dp = {0: []}
        queue = deque([0])

        while queue:
            state = queue.popleft()
            guards = dp[state]

            if state & full == full:
                return guards

            for v, cov in enumerate(self.coverage_sets):
                new_state = state
                for r in cov:
                    new_state |= (1 << r)

                if new_state not in dp or len(dp[new_state]) > len(guards)+1:
                    dp[new_state] = guards + [v]
                    queue.append(new_state)
```

13.5 Implementação da Extensão Guardas Coloridos

```
def build_conflict_graph(selected_guards, coverage_matrix):

    graph = {
        g: set()
        for g in selected_guards
    }

    n_rectangles = len(coverage_matrix[0])

    for r in range(n_rectangles):

        guards_covering = [
            g
            for g in selected_guards
            if coverage_matrix[g][r] == 1
        ]

        for i in range(len(guards_covering)):
            for j in range(i + 1, len(guards_covering)):

                g1 = guards_covering[i]
                g2 = guards_covering[j]

                graph[g1].add(g2)
                graph[g2].add(g1)

    return graph

def exact_coloring(graph):

    nodes = list(graph.keys())

    # order nodes by degree (important for pruning)
    nodes.sort(key=lambda x: len(graph[x]), reverse=True)

    n = len(nodes)

    best_coloring = {}
    best_k = float("inf")

    coloring = {}

    def backtrack(i, used_colors):

        nonlocal best_coloring, best_k
```

```
# pruning
if used_colors >= best_k:
    return

if i == n:
    best_coloring = coloring.copy()
    best_k = used_colors
    return

node = nodes[i]

forbidden = {
    coloring[nbr]
    for nbr in graph[node]
    if nbr in coloring
}

for c in range(used_colors):

    if c not in forbidden:
        coloring[node] = c
        backtrack(i + 1, used_colors)
        del coloring[node]

# try new color
coloring[node] = used_colors
backtrack(i + 1, used_colors + 1)
del coloring[node]

backtrack(0, 0)

return best_coloring, best_k
```

13.6 Implementação da Extensão Alcance D

```
def build_rectangle_graph(instance):  
  
    n = len(instance.rectangles)  
  
    graph = {  
        i: set()  
        for i in range(n)  
    }  
  
    for i in range(n):  
  
        ri = set(instance.rectangles[i].vertices)  
  
        for j in range(i + 1, n):  
  
            rj = set(instance.rectangles[j].vertices)  
  
            # adjacent if share at least one vertex  
            if ri & rj:  
  
                graph[i].add(j)  
                graph[j].add(i)  
  
    return graph  
  
def bfs_expand(rect_graph, start_rectangles, D):  
  
    visited = set(start_rectangles)  
  
    queue = deque()  
  
    for r in start_rectangles:  
        queue.append((r, 0))  
  
    while queue:  
  
        node, dist = queue.popleft()  
  
        if dist == D:  
            continue  
  
        for neigh in rect_graph[node]:  
  
            if neigh not in visited:
```

```

        visited.add(neigh)

        queue.append((neigh, dist + 1))

    return visited

# =====
# Expanded coverage sets for distance D
# =====

def build_expanded_coverage_sets(instance, D):

    rect_graph = build_rectangle_graph(instance)

    vertex_index = {
        v: i
        for i, v in enumerate(instance.vertices)
    }

    coverage_sets = [
        set()
        for _ in instance.vertices
    ]

    # direct coverage
    for rect_idx, rect in enumerate(instance.rectangles):

        for v in rect.vertices:

            vi = vertex_index[v]

            coverage_sets[vi].add(rect_idx)

    # expand with BFS
    expanded = []

    for cov in coverage_sets:

        expanded_cov = bfs_expand(
            rect_graph,
            list(cov),
            D
        )

        expanded.append(sorted(list(expanded_cov)))

    return expanded

```

14. Algoritmo de Benchmarking

Para medir o desempenho médio dos algoritmos implementados, foi utilizado o script `performance_benchmark.py`, baseado em execução isolada por processo e limitação temporal por instância.

14.1 Estratégia de Medição

A metodologia adotada segue os passos:

1. Ler as instâncias de cada ficheiro de entrada;
2. Construir, para cada instância, os conjuntos de cobertura e a matriz binária de cobertura A ;
3. Executar cada solver em processo separado (`multiprocessing.Process`);
4. Impor limite máximo de execução de 60 segundos por execução;
5. Repetir várias execuções e calcular o tempo médio com `statistics.mean`.

A execução isolada por processo permite interromper algoritmos que excedam o tempo máximo sem bloquear o benchmarking global.

14.2 Pseudocódigo

```
Benchmark(files, solvers, runs, time_limit):  
  para cada file em files:  
    instances <- parse_instances(file)  
  
    para cada solver em solvers:  
      times <- []  
      timeout <- falso  
  
      repetir runs vezes:  
        para cada instância i em instances:  
          coverage_sets, vertices <- build_coverage_sets(i)  
          A <- build_matrix(coverage_sets, vertices, i.rectangles)  
  
          t <- run_with_timeout(solver, coverage_sets, A, i, time_lim  
it)  
  
          se t == TIMEOUT:  
            timeout <- verdadeiro  
            parar  
            adicionar t a times  
  
      se timeout:  
        escrever "time limit exceeded"  
      senão:  
        escrever mean(times)
```

14.3 Implementação Utilizada

```

TIME_LIMIT = 60

def time_solver(solver_func, coverage_sets, A, i, queue):
    try:
        result = solver_func(coverage_sets, A, i)
        queue.put(("ok", result))
    except Exception as e:
        queue.put(("err", repr(e)))

def benchmark_one(solver_func, coverage_sets, A, i):
    queue = mp.Queue()
    p = mp.Process(target=time_solver, args=(solver_func, coverage_sets, A, i,
    queue))

    start = time.perf_counter()
    p.start()
    p.join(TIME_LIMIT)

    if p.is_alive():
        p.terminate()
        p.join()
        return None

    end = time.perf_counter()
    return end - start

def benchmarker(instances, solver_func, runs=2):
    instance_times = []

    for _ in range(runs):
        for i in instances:
            coverage_sets, vertices = build_coverage_sets(i)

            A = [[0 for _ in i.rectangles] for _ in vertices]
            for vi, cov in enumerate(coverage_sets):
                for r in cov:
                    A[vi][r] = 1

            t = benchmark_one(solver_func, coverage_sets, A, i)
            if t is None:
                return None

            instance_times.append(t)

```

```
return mean(instance_times)
```

Esta implementação foi a base dos resultados apresentados no Capítulo 10.

15. Discussão Técnica Global

A implementação prática confirmou que toda a arquitetura do projeto pode ser construída em torno de uma única abstração central: a matriz de cobertura entre vértices e retângulos.

Esta decisão revelou-se particularmente vantajosa porque permitiu reutilizar exatamente os mesmos dados de entrada em todos os paradigmas de resolução estudados.

Em termos computacionais observou-se:

- Greedy: excelente escalabilidade;
- OR-Tools: melhor compromisso entre optimalidade e desempenho;
- MAC+AC3: implementação académica rica e eficiente;
- DP: forte valor teórico mas escalabilidade limitada.